Research Computing: Lecture 1

Linux Basics — Study Notes

Prepared for J. Xie

Source: Philip Blakely, Centre for Scientific Computing, University of Cambridge, `Handout_1_full.pdf`, PDF pages 1–37. The footer numbers these within a larger 166-slide course. These notes paraphrase the supplied handout; the personalised setup advice, clarifications, and practice exercise are added study aids.

Contents

1 Purpose and your setup (pp. 3–6) **1**

2 Linux and the Unix philosophy (pp. 7–10) **2**

3 Prompts, paths, and directories (pp. 11–14) **2**

4 Files, help, and typing (pp. 15–19) **3**

5 Identity and shared resources (pp. 20–21) **3**

6 Standard streams and redirection (pp. 22–26) **4**

7 Pipes and text tools (pp. 25–28) **4**

8 Manipulate, search, compare (pp. 29–30) **4**

9 Permissions and groups (pp. 31–34, 36) **5**

10 Symbolic links and file types (pp. 35–37) **5**

11 Practice workflow (added study exercise) **6**

12 Revision questions and answers **6**

1 Purpose and your setup (pp. 3–6)

The lecture introduces the Linux command line. Later course topics include text processing, networking, scripting, compiling/linking, Makefiles, and version control.

Your Mac already has a Unix-like terminal and Homebrew. Homebrew installs software; it does not turn macOS into Linux. Apollo is a remote Linux computer. A terminal in VS Code's SSH window

operates on Apollo's files, whereas a local terminal operates on your Mac. Your Desktop folder and remote home folder are separate.

```
# On your Mac: connect using your configured SSH alias
ssh cambridge-csc
# On Apollo: check who and where you are
whoami
hostname
pwd
# Leave the SSH session
exit
```

The SSH commands are personalised setup context. Keep Cambridge VPN connected when required to reach CSC from outside the University network. The handout stresses understanding commands and agent-generated instructions before running them: consider effects on your data and shared resources.

2 Linux and the Unix philosophy (pp. 7–10)

Linux technically means the kernel: the interface between applications and hardware. **GNU/Linux** combines it with tools and other software. A **distribution** packages a complete operating system. The handout lists Ubuntu (CSC), Debian (Ubuntu's basis), Red Hat, Fedora, and Rocky Linux (CSD3). Skills largely transfer between distributions.

The Unix philosophy is to combine small programs with focused jobs, using text streams to connect them. A pipeline lets several simple tools perform a complex task.

Linux also has desktop applications: the slides mention Firefox, Thunderbird, LibreOffice, GIMP, and Inkscape. This lecture focuses on the terminal.

3 Prompts, paths, and directories (pp. 11–14)

```
jx355@apollo:~$
```

This identifies user `jx355`, machine `apollo`, and the home directory `~`. The dollar sign is the prompt: do not type it with commands.

Command	Meaning
`ls`	List current directory contents.
`pwd`	Print your current directory's full path.
`cd Documents`	Enter a directory relative to your current location.
`cd` or `cd ~`	Return home.
`cd ..`	Move to the parent directory.
`cd -`	Return to your previous working directory.
`mkdir MPhil`	Create a directory.
`mkdir -p MPhil/Assignment1`	Create a directory and missing parents.

Absolute paths start at the root directory `/`; **relative paths** start from the current directory. `.` means the current directory and `..` its parent. Directories form a tree; slashes separate path components.

Root `/` differs from your home directory and from the administrator account named `root`. Common locations: `/usr/bin` for executables, `/usr/lib` for libraries, and `/opt` or `/usr/local` for additional

software. CSC uses `/lsc/opt`. Your home path may differ from `/home/jx355`; run `cd` then `pwd` to find it.

4 Files, help, and typing (pp. 15–19)

```
cat notes.txt      # Display text contents
man ls             # Read a command's manual
ls -a              # Include hidden entries
ls -l              # Detailed listing
ls -al             # Same options combined
man 3 printf       # Specify a manual section
```

`cat` joins/displays file contents to stdout. Avoid displaying binary executables as text: the output can disrupt your terminal.

In manual pages, navigate with arrows, search with `/`, and quit with `q`. Manual sections distinguish command documentation from, for example, library functions of the same name. Options commonly begin with `-`; some short options combine.

Use up/down arrows to recall commands and Tab to complete names. In the Bash behaviour described by the slides, pressing Tab twice lists alternatives. Your Mac's default zsh may behave differently.

Spaces separate command arguments. Prefer filenames without spaces, or quote/escape them:

```
cat "My Data.dat"
cat My\ Data.dat
```

The slides introduce gedit, Emacs, Vim, Geany, Sublime Text, and VS Code. Your VS Code Remote SSH window edits remote files. The lecture links CSC's VS Code guidance, including home-quota considerations.

5 Identity and shared resources (pp. 20–21)

```
whoami     # Current username
groups     # Group memberships
id         # User ID, group ID, memberships
passwd     # Change your password interactively
who        # Currently logged-in users
last       # Recent login history
top        # Processes/resource use; press q to quit
```

The handout asks users to change their initial password. Run `passwd` on Apollo and follow its prompts privately. The CSC login password differs from the passphrase used to reveal it in the email's secret link.

Many users share the machine: avoid excessive CPU, memory, and disk activity. The slides recommend lowering priority for long-running commands. An explicit spelling of their `nice -19` example is:

```
nice -n 19 ./myProgram
```

A larger niceness value means lower scheduling priority. This does not cap memory or runtime; follow CSC's separate rules for compute-intensive jobs.

The lecture compares load average with CPU core count. A clarification: Linux load includes runnable tasks and tasks in uninterruptible sleep (often waiting for I/O); it is not a CPU percentage.

6 Standard streams and redirection (pp. 22–26)

Stream	Descriptor	Default
stdin	0	Input from terminal
stdout	1	Normal output to terminal
stderr	2	Diagnostics/errors to terminal

Redirection makes repeated runs easier by taking inputs from files and saving outputs.

```
./myProgram < myInput > myOutput
./myProgram < myInput >> myOutput
./myProgram < myInput > myOutput 2> myErrors
./myProgram < myInput > combined.log 2>&1
```

`<` supplies stdin. `>` creates or overwrites output; `»` appends. `2>` redirects stderr. `2>&1` sends stderr to stdout's current destination; order matters, so place it after the file redirection to combine both streams there. The slides also show Bash shorthand `>&`; the explicit form is clearer.

Remember: `>` can erase existing contents when the shell opens the file, even if the program later fails.

7 Pipes and text tools (pp. 25–28)

A pipe `|` connects one command's stdout to another's stdin; stderr is normally separate.

```
cat Makefile | wc
wc < Makefile
wc Makefile
```

These count the same contents, but the final version also prints the filename. Default `wc` reports lines, words, and bytes. The slides call the last count characters; bytes and characters can differ for Unicode. Use `wc -l` for lines and `wc -m` for characters.

`sort` sorts lines. `uniq` removes repeated *adjacent* lines. To remove repeated lines wherever they occurred, sort first. `spell` identifies possible spelling errors, where installed; `detex` can remove LaTeX markup first.

```
spell Project_1.tex | sort | uniq
spell Project_1.tex | sort | uniq >> Project_1_typos
./myProgram 2>&1 | tee myProgram.out | ./postProcess.sh
```

`tee` copies input to both a file and stdout, so a pipeline can continue. It overwrites by default; `tee -a` appends. The slides use Bash `|&` to pipe stdout and stderr together; the example above writes this explicitly. The processing script must exist and be executable.

8 Manipulate, search, compare (pp. 29–30)

```
cp Thesis.tex Thesis_old.tex
cp -r Thesis Thesis_old
mv Thesis.tex Chapter1.tex
rm unwanted.txt
ls *.pdf
grep -r "FIXME" ./MPhil_SourceCode
find ./MPhil -name "MyPlot.png"
diff Thesis_1.tex Thesis_2.tex
```

`cp` copies; `cp -r` copies a directory recursively. `mv` moves/renames. `rm` removes. Copies/moves can overwrite existing destinations.

Wildcards: `*` matches zero or more characters; `?` matches one. `simOutput?.png` matches `simOutput1.png`, but not `simOutput13.png`. The shell expands unquoted wildcards before running the command. Quote a pattern in `find -name '*.png'` so `find` interprets it.

`grep` searches file contents; `find` searches filesystem entries. `diff` compares text; normal output uses `<` for lines from the first file and `>` for the second.

The lecture shows `rm -rf` for forced recursive removal. Terminal deletion normally bypasses Trash. Preview wildcard matches with `ls` before deleting anything.

9 Permissions and groups (pp. 31–34, 36)

```
-rw-r--r--  1 jx355 csc 9628 Oct 8 notes.txt
drwxr-x---  2 jx355 csc 4096 Oct 8 coursework
```

The initial character gives type: `-` regular file, `d` directory, `l` symbolic link. The following nine characters form three triplets: owner/user, group, others.

For a file, `r` means read, `w` write, `x` execute. For a directory, `r` permits listing names, `x` permits traversal, and `w` permits changing entries (usually also requiring `x`). Directory semantics are an explanatory addition. Access depends on directories along the path; deletion depends chiefly on the parent directory's permissions.

`-rw-r-r-`: owner reads/writes; group and others read. `-rwxr-xr-x`: owner has all permissions; group and others read/execute.

```
chmod go-r myFile       # Remove group/others read access
chmod u+x myScript     # Add owner execute access
chgrp mygroup myFile   # Change group, if permitted
```

In symbolic `chmod`, `u/g/o` select owner/group/others, `+/-` add/remove, and `r/w/x` select permissions. `go-r` only removes read permission; any other permissions remain.

The handout says CSC defaults make newly created files readable/writable only by the owner. Defaults vary across systems: inspect actual permissions with `ls -l` rather than assuming every existing file is private.

10 Symbolic links and file types (pp. 35–37)

A symbolic link refers to another path rather than making a copy. Editing via the link affects its target. Removing the link itself leaves the target; moving/removing the target can break the link.

```
ln -s /path/to/existing/data data_link
ls -l data_link
file image.png
file /usr/bin/gcc
```

The first example is schematic: substitute a real target. Order is target, then link name. Linux does not rely solely on extensions for file types or executability. `file` inspects contents to suggest a type. Renaming a file does not convert its format.

11 Practice workflow (added study exercise)

Run on Apollo in a fresh practice directory. If this directory already exists, choose a different name to preserve your work.

```
cd ~
mkdir lecture1-practice
cd lecture1-practice
pwd
printf 'banana\napple\nbanana\n' > words.txt
cat words.txt
wc -l words.txt
sort words.txt | uniq > unique_words.txt
cat unique_words.txt
cp words.txt words_backup.txt
diff words.txt words_backup.txt
printf 'pear\n' >> words.txt
diff words_backup.txt words.txt
ls -l
chmod go-r words_backup.txt
ls -l words_backup.txt
ln -s words.txt words_link.txt
cat words_link.txt
find . -name '*.txt'
grep 'banana' words.txt
```

Expected results: original line count 3; unique sorted output `apple`, then `banana`; first comparison no output; second comparison shows an added `pear` line. Reading through the link shows the updated original. Keep the files for revision.

12 Revision questions and answers

1. Absolute versus relative path? Root-based versus current-directory-based.
2. `>` versus `»`? Overwrite versus append.
3. Why sort before `uniq`? It only removes adjacent repeated lines.
4. `grep` versus `find`? Contents versus filesystem entries.
5. Decode `-rwxr-x--`. Regular file; owner read/write/execute; group read/execute; others none.
6. Symbolic link versus copy? Reference to a target versus an independent file.
7. Where do Remote SSH terminal commands run? On the remote server, Apollo in your CSC session.